CSCD602
Advanced Software Engineering

Session 6 – Software Effort Estimation

By
Solomon Mensah (PhD)
Dept. of Computer Science
smensah03@ug.edu.gh

UNIVERSITY OF GHANA

Session Outline

- What is Software Effort Estimation, and why it matters
- Types of estimation techniques: expert judgment, algorithmic models, analogy, ML, agile
- COCOMO and Function Point Analysis in depth, with a worked example
- The estimation process, key influencing factors, and common challenges
- Estimation biases and best practices

Learning Objectives

- **By the end of this session, students should be able to:**
  - Define software effort estimation and distinguish effort, cost, duration, and price.
  - Explain why accurate estimation matters to projects and organisations.
  - Compare estimation techniques: expert judgment, algorithmic models, analogy, ML, and agile methods.
  - Apply the COCOMO model to compute effort for a sample project.
  - Identify factors, biases, and challenges that affect estimation accuracy.
  - Apply best practices to produce more reliable estimates.

Where Estimation Fits In

- Estimation is a core Planning activity, but it also feeds Initiation and Monitoring.
  - Initiation — rough, order-of-magnitude estimate for feasibility and bids.
  - Planning — detailed effort, cost, and schedule estimation.
  - Execution — the team works against the estimated plan.
  - Monitoring & Controlling — actuals compared to estimates; re-estimate as needed.
  - Closure — actuals recorded as historical data for future estimates.

What Is Software Effort Estimation?

- The process of predicting the effort (person-hours/person-months) required to design, build, test, and deliver a software system.
  - Effort — total labour required.
  - Cost — budget required, derived from effort × rate.
  - Duration — calendar time (schedule) to complete the work.
- In short: “How much work, time, and money will this project take?” It is a forecast, not an exact figure.

Effort, Cost, Duration & Price

- Effort — total labour: person-months or person-hours of work.
- Duration — elapsed calendar time from start to finish.
- Cost — Effort × labour rate, plus overheads (tools, infrastructure).
- Price — Cost + profit margin — what the client is charged.
- Cost = Effort × Rate per person-month
- Price = Cost + Profit Margin + Contingency

Why Estimate Effort or Cost?

- Budgeting — determine funding needed before committing resources.
- Scheduling — set realistic timelines and delivery dates.
- Resource Allocation — decide staffing levels and required skills.
- Client Trust — support contracts, bids, and client negotiations.
- Risk Management — identify risks early and plan contingencies.
- Progress Tracking — Provides a baseline against which performance can be measured, monitored, and evaluated over time to assess progress toward established goals and objectives.

Consequences of Poor Estimation

- **Underestimation:**
  - Missed deadlines, rushed delivery, and team burnout.
  - Compromised quality, more defects, and budget overruns.
  - Damaged trust with stakeholders.
- **Overestimation:**
  - Wasted budget/resources and lost bids to competitors.
  - Reduced productivity (“Parkinson's Law”) and missed opportunities.

The Cone of Uncertainty

- Estimation accuracy improves as the project progresses and more is known.
- Early estimates carry the widest error range — communicate a range, not a false-precision number.

Project Stage	Typical Estimate Range
Initial Concept	0.25x – 4x
Requirements Defined	0.5x – 2x
Design Complete	0.67x – 1.5x
Development Underway	0.8x – 1.25x
Testing	0.9x – 1.1x

Types of Effort Estimation Techniques

- Expert Judgment — experienced professionals estimate using intuition and past project knowledge.
- Algorithmic Models — formulas using project size metrics (COCOMO, Function Points).
- Estimation by Analogy — compare with similar, previously completed projects.
- Machine Learning-Based — learn from historical data using regression or neural networks.
- Agile / Relative Sizing — story points and planning poker.

Expert Judgment

- One or more experienced people estimate effort based on knowledge of similar past projects and intuition.
- Best when historical data is limited, the project is novel, or a quick sanity check is needed.
- Watch out for personal bias, overconfidence, and inconsistency between experts.
- **Common forms:**
  - Single-expert judgment, group consensus, Wideband Delphi, comparison to a mentor's project.

The Delphi Technique

- A structured way to combine expert opinions while reducing dominance, groupthink, and anchoring.
  - Each expert estimates independently and anonymously.
  - Estimates and reasoning are shared with the group.
  - Experts discuss and revise their estimates.
  - Repeat rounds until estimates converge.

Top-Down vs Bottom-Up

- Top-Down — estimate the whole project first, then allocate portions to phases/modules.
  - Fast, useful for early bids; less accurate for complex, novel work.
- Bottom-Up — break into small tasks, estimate each, then sum up.
  - More accurate and defensible; needs a detailed task breakdown (WBS).

WBS-Based Estimation

- A work-breakdown structure (WBS) decomposes a project into progressively smaller, manageable components — the foundation for bottom-up estimation.
  - Decompose scope into modules, then tasks, until each is small enough to estimate confidently.
  - Estimate effort for each leaf-level task, then sum upward for the total.
  - Improves traceability and makes it easy to track progress against the plan.

Three-Point Estimation (PERT)

- Optimistic (O) — best-case effort if everything goes smoothly.
- Most Likely (M) — realistic effort under normal conditions.
- Pessimistic (P) — worst-case effort if problems arise.
- Expected Effort (E) = (O + 4M + P) / 6
- Example: O=20, M=30, P=50 → E = (20+120+50)/6 ≈ 31.7 person-days

Algorithmic Models: An Overview

- Algorithmic models turn measurable project attributes into an effort figure using a calibrated formula.
  - COCOMO / COCOMO II — effort from estimated lines of code (KLOC).
  - Function Point Analysis — effort from counted functionality.
  - Use Case Points — effort from use case/actor complexity.
  - Putnam / SLIM — effort and schedule from a staffing curve.

COCOMO — Basic Model

- Constructive Cost Model (Barry Boehm, 1981) estimates effort from estimated size in KLOC.
- Effort = $a \times (KLOC)^b$ (person-months)
  - Effort: The total labor required, measured in Person-Months (PM).
  - a: A constant coefficient based on the project's development mode.
  - KLOC: Kilo Lines of Code (thousands of lines of source code).
  - b: An exponent that represents the economy of scale as project size increases
  - a and b are empirical constants depending on project type (organic, semi-detached, embedded).
  - Quick to apply, but requires an early code-size estimate and ignores team/tool factors.

COCOMO — Project Types & Modes

- Higher a and b values reflect the extra effort demanded by complexity and constraints.

Mode	Characteristics	a	b
Organic	Small, familiar team; flexible requirements	2.4	1.05
Semi-detached	Medium size/complexity; mixed experience	3.0	1.12
Embedded	Complex, tightly constrained (safety, hardware)	3.6	1.20

Function Points — Concept

- Measures software size by counting functionality delivered to the user — independent of programming language.
- Can be counted directly from requirements/design documents, before code is written.
- Advantages: usable earlier in the lifecycle, comparable across technologies, easy to justify to clients.

Function Points — Calculation

- Components: External Inputs, External Outputs, External Inquiries, Internal Logical Files, External Interface Files.
- Steps: count each component → rate complexity → apply weights (Unadjusted FP) → apply Value Adjustment Factor → Adjusted FP.

Component	Simple	Average	Complex
External Input	3	4	6
External Output	4	5	7
External Inquiry	3	4	6
Internal Logical File	7	10	15

Function Points — Calculation

- Components: External Inputs, External Outputs, External Inquiries, Internal Logical Files, External Interface Files.
- Steps: count each component → rate complexity → apply weights (Unadjusted FP) → apply Value Adjustment Factor → Adjusted FP.

Component	Average	Complex
External Input	4	6
External Output	5	7
External Inquiry	4	6
Internal Logical File	10	15

Use Case Points (UCP)

- Estimates effort from the complexity of a system's actors and use cases.
  - Weighted Actors — rated Simple/Average/Complex by interaction type.
  - Weighted Use Cases — rated by number of transactions/steps.
  - Technical & Environmental Factors — adjust for performance, reusability, team experience.
- Effort (hours) = UCP × Productivity Factor (commonly ~20 hrs/UCP)

Estimation by Analogy

- Compare the new project to similar, previously completed projects with known actual effort.
  - Find similar projects.
  - Compare attributes (size, complexity, technology, team).
  - Adjust for differences.
  - Derive the estimate.

Machine Learning Estimation

- Models trained on historical project data learn to predict effort for new projects.
  - Approaches: regression, decision trees/random forests, neural networks, case-based reasoning.
  - + Captures complex, non-linear patterns; improves with more data.
  - − Needs a sizeable dataset; can be a “black box” to stakeholders.

Agile Estimation — Story Points

- Story points are a relative measure of effort, complexity, and risk — not a direct measure of time.
  - Scaled using a Fibonacci-like sequence: 1, 2, 3, 5, 8, 13, 21 …
  - Planning Poker: read the story → each member privately picks a card → reveal together → discuss outliers and re-vote.

Worked Example: COCOMO

- Scenario: an organic-mode project estimated at 10,000 lines of code (10 KLOC).
  - Effort = a × $(KLOC)^b$; organic mode: a = 2.4, b = 1.05
  - Effort = 2.4 × (10)^1.05
  - (10)^1.05 ≈ 11.22
  - Effort = 2.4 × 11.22 ≈ 26.9 person-months
- With 5 developers, duration ≈ 26.9 ÷ 5 ≈ 5.4 months (simplified).

The Effort Estimation Process

- 1. Understand project scope and requirements.
- 2. Break work into smaller, manageable tasks (WBS).
- 3. Choose one or more estimation techniques.
- 4. Estimate effort for each task.
- 5. Aggregate and validate the total estimate.
- 6. Track actuals and refine future estimates.

Factors Affecting Estimation

- Project Size & Complexity — larger, more intricate systems require more effort.
- Team Experience & Skill — experienced teams deliver faster with fewer errors.
- Requirements Volatility — frequent change increases rework and uncertainty.
- Tools & Technology — mature tools and reusable components reduce effort.
- Process & Methodology — Agile, Waterfall, or hybrid approaches.
- Quality Requirements — stricter reliability/security/performance needs raise effort.

Common Estimation Challenges

- Changing Requirements — scope creep invalidates earlier estimates.
- Human Bias — optimism or stakeholder pressure skews estimates.
- Lack of Historical Data — new teams have little to compare against.
- Unclear Requirements — ambiguity makes accurate sizing difficult.
- Technology Uncertainty — unfamiliar tools add unpredictable effort.
- External Pressure — deadlines fixed before estimation distort the numbers.

Biases & Best Practices

- **Common biases:**
  - Planning fallacy, anchoring, optimism bias, pressure bias.
- **Best practices:**
  - Use multiple techniques and compare (triangulation).
  - Break work down before estimating (WBS).
  - Estimate as a range, not a single number.
  - Record actuals and review estimation accuracy after each project.

Key Takeaways

- Effort estimation predicts the work, time, and cost needed for a software project.
- It underpins budgeting, scheduling, staffing, and risk management.
- Accuracy naturally improves as a project progresses (Cone of Uncertainty).
- Techniques include expert judgment, algorithmic models, analogy, ML, and agile methods.
- COCOMO and Function Points are the classic algorithmic models — know their formulas.
- No single method is perfect — combine techniques and track actuals to improve over time.

Thank you